BSc (Hons) in **Computer Science**

Faculty of Computing

SE3062 : Intelligent Systems

Year 3 · Semester 1 · 2026 Semester 2

Practical 01

> **Objective & Lecture Mapping:** Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation

Task 0 : Setting up and getting the starter Code

1. Go to the Git Hub Repo. Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (__init__)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

1. Performance measure
2. Environment
3. Actuators
4. Sensor

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

1. self.width
2. self.height
3. self.walls
4. self.food_positions
5. self.opponents

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

Multi-Agent

The self.opponents variable is initialized as a list of opponent positions, and during execute_action(), each opponent chooses a random action and moves. So multiple autonomous entities act within the same environment.

Task 1.2: The Perception Subsystem (get_percept)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

The absolute, cumulative historical log of every single percept the agent has received from its initialization up to the current moment.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

Sensors

6. (Evaluate) Based on the exact dictionary returned by `get_percept()`, is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

Partially observable

Since the percept does not reveal the complete environment state, the agent must explore and remember previously visited locations to find the remaining food.

Task 1.3: Action Execution (execute_action)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

Performance measure

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

Difference between external results and internal processing effort.

Measuring external results:

1. Focuses on the actual impact of the agent's actions.
2. The metric reflects whether the agent is behaving successfully according to the task goal.
3. It is independent of the agent's implementation details.

Measuring internal processing effort:

1. Focuses on how the agent thinks rather than what it achieves.
2. Two agents may use different amounts of computation but achieve the same result.

A performance measure should evaluate observable outcomes, not the internal mechanism used to produce them. An efficient agent and a complex agent should be compared based on their success in the environment, such as avoiding walls, collecting food, and achieving a high score.

Part 2: Guided Modification & Deep Theory Mapping

> Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (__init__)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally hide this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

Become more partially observable.

Because the exact coordinates of active hazards are completely omitted from the agent's percept vector, the agent cannot determine the safety of adjacent cells from its sensors alone.

Step 2.2: Updating the Perception Subsystem (get_percept)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

With smells_toxin, the agent can:

1. Detect immediate danger before taking the next action.
2. Avoid toxic cells to reduce negative rewards.
3. Choose safer paths while still trying to collect food.
4. Balance the reward of collecting food against the risk of encountering traps.

Step 2.3: Modifying Action Execution & Visual Rendering (execute_action & draw_grid)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

If we give the agent +1 point every time it activates its suction motor and collects a unit of dirt. A mathematically rational agent will find the shortest path to maximize this metric. It will clean a patch of dirt, purge its container back onto the floor, and clean the exact same spot repeatedly forever to harvest infinite score points.

Finalize and Lock Lab 01 Analytical Evaluation



FACULTY OF COMPUTING